

Zadanie 1

Napisz program wczytujący z konsoli w pętli (**while**) linie zawierające jedną, dwie lub trzy liczby. Dla każdej wczytanej linii utwórz krotkę zawierającą trzy elementy

- Napis "S" (square) jeśli wczytana została jedna liczba, lub "R" (rectangle) jeśli dwie, lub "C" (cuboid) jeśli trzy;
- krotkę zawierającą trzy wczytane liczby;
- liczbę będącą wartością powierzchni/objętości kwadratu, prostokąta, lub prostopadłościanu zdefiniowanego wczytanymi liczbami.

Utworzone krotki umieszczaj w liście. Pętla kończy się, gdy użytkownik wpisze pustą linię (poprzez wciśnięcie ENTER) i wtedy lista jest drukowana.

Uruchomienie programu mogłoby zatem wyglądać tak:

```
Enter sizes: 3
Enter sizes: 4 5 1
Enter sizes: 3 4
Enter sizes:
[('S', (3,)), 9), ('C', (4, 5, 1), 20), ('R', (3, 4), 12)]
```

Wskazówka: użyj metody **split** klasy **str** do wyodrębnienia liczb z wczytanej linii.

Zadanie 2

Pewna liczba aktorów (ponumerowanych od 1) występowało w filmach o tytułach 'a', 'b', 'c', itd. W słowniku **films** kluczami są tytuły filmów a wartościami listy aktorów w nich występujących.

Napisz funkcję która przyjmuje taki słownik, a zwraca słownik, w którym aktorzy (ich numery) są kluczami, a listy filmów w których wystąpili są wartościami.

Wypisz zawartość wynikowego słownika w kolejności według liczby filmów malejąco, a jeśli liczba filmów jest taka sama, to według numeru przypisanego aktorom rosnąco.

Na przykład dla słownika

```
films = dict(
    a=[1, 3, 5], b=[2, 1, 4], c=[5, 2], d=[3, 4, 6],
    e=[1], f=[3, 4, 5], g=[1, 7], h=[7, 6]
)
```

wynik powinien wyglądać tak

```
1 -> ['a', 'b', 'e', 'g']
3 -> ['a', 'd', 'f']
4 -> ['b', 'd', 'f']
```

```
5 -> ['a', 'c', 'f']
2 -> ['b', 'c']
6 -> ['d', 'h']
7 -> ['g', 'h']
```

Zadanie 3

Napisz funkcję pobierającą dowolną listę, a zwracającą krotkę zawierającą wartość elementu listy, który powtarzał się w niej największą liczbę razy, oraz tę liczbę powtórzeń (w razie remisu funkcja zwraca dowolne z rozwiązań).

Na przykład dla

```
lst = [-4, 0, 3, 2, -4, -3, 3, 2, -4]
```

odpowiedź powinna być (-4, 3), a dla

```
lst = 'to be or not to be'.split()
```

albo ('to', 2) albo ('be', 2).

Użyj tylko jednej pętli! Nie używaj **collections.Counter**.

Zadanie 4

Dany jest słownik osób należących do towarzystwa miłośników książek, na przykład

```
bibliophiles = {
    'John': {1, 2, 7, 11, 29},
    'Mary': {7, 9, 11, 5},
    'Jane': {1, 4, 5},
    'Bill': {7, 11, 1, 2, 9},
    'Kate': {4, 5, 9, 11}
}
```

Kluczami są ich imiona, wartościami zbiory posiadanych przez nich (i przeczytanych!) książek jako zbiór identyfikatorów w postaci liczb naturalnych.

● 1. Napisz funkcję przyjmującą tego rodzaju słownik oraz imię jednego z członków towarzystwa, a zwracającą listę trzech elementów:

1. Imię osoby, z którą dany członek towarzystwa ma najwięcej wspólnych książek;
2. Liczbę wspólnie posiadanych książek;
3. Zbiór książek, które znaleziona osoba mogłaby polecić (tych, które ona ma, ale nie ma ich osoba podana jako argument funkcji).

Na przykład jeśli do funkcji przesłany został podany wyżej słownik i napis 'John', powinniśmy otrzymać

```
['Bill', 4, {9}]
```

a dla 'Mary'

```
['Bill', 3, {1, 2}]
```

● **2.** Napisz funkcję przyjmującą tego rodzaju słownik oraz imię jednego z członków towarzystwa, a zwracającą krotkę dwóch elementów:

1. Identyfikator najbardziej popularnej książki spośród tych, które dana osoba posiada;
2. Liczbę innych osób, które ją posiadają.

Na przykład jeśli do funkcji przesłany został podany wyżej słownik i napis `'John'`, powinniśmy otrzymać

```
(11, 3)
```

a dla `'Jane'`

```
(1, 2)
```

● **3.** Napisz funkcję przyjmującą tego rodzaju słownik oraz imię jednego z członków towarzystwa, a zwracającą krotkę dwóch elementów:

1. Identyfikator najbardziej popularnej książki spośród tych, których dana osoba *nie* posiada;
2. Liczbę innych osób, które ją posiadają.

Na przykład jeśli do funkcji przesłany został podany wyżej słownik i napis `'John'`, powinniśmy otrzymać

```
(9, 3)
```

a dla `'Kate'`

```
(1, 3)
```

Zadanie 5 _____

Napisz funkcję, która pobiera listę słów a zwraca słownik w którym wartościami są listy słów z tego słownika będące anagramami tego samego słowa (czyli różniące się tylko kolejnością liter).

Z wynikowego słownika wypisz najdłuższą listę anagramów.

Jeśli wykorzystasz plik angielskich słów [englishWords.txt](#) (jest tam ponad 100 000 słów), wynikiem powinna być lista

```
['apers', 'apres', 'asper', 'pares', 'parse',  
'pears', 'rapes', 'reaps', 'spare', 'spear']
```

Aby przeczytać plik do listy zawierającej jego linie (bez znaku EOL na końcu) możesz użyć następującej konstrukcji

```
from pathlib import Path
...
with open(Path.home() / 'englishWords.txt') as f:
    list_of_words = [line.rstrip() for line in f]
```

zakładając, że plik *englishWords.txt* znajduje się w Twoim katalogu domowym.
